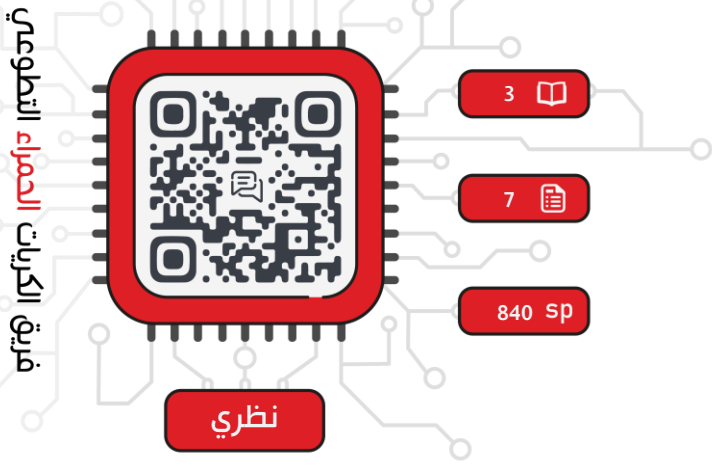


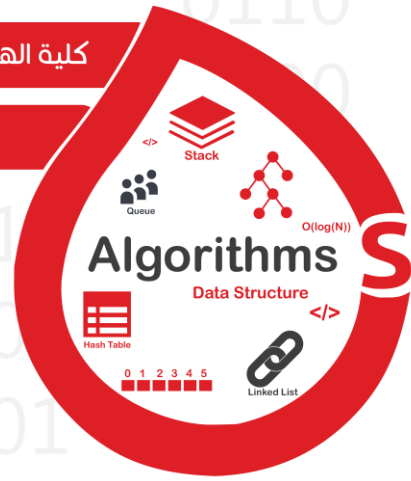


كلية الهندسة المعلوماتية

السنة الثانية

مقدمة عن طرق التفكير

د. أبي صندوق



RB Informatics; 02/11/2025

خوارزميات 1

طرق التفكير

في مادة الخوارزميات نبحث عن حل لمسألة معيّنة و هذا يستدعي التفكير بطرق مناسبة لكل مسألة.
و طرق التفكير هي مجموعة أدوات أساسية "standards" تساعد بحل نسبة معينة من المسائل.

أولاً Decrease and Conquer

نفترض أنك تريد صعود سلم من 10 درجات فمن الواضح أنك لن تستطيع صعوده بخطوة واحدة، لذلك تصعد درجة درجة.

إذاً يصبح الأمر أسهل بكثير عند **تقسيم السلم إلى مجموعة خطوات أصغر.**

إن هذه الطريقة في التفكير لهذا النوع من المسائل تدعى **Decrease and Conquer**.

(التقليل و التغلب) Decrease and Conquer

هو نموذج خوارزميات حيث:

1. يتم **تقليل** حجم المشكلة تدريجياً في كل خطوة تكرارية.

2. يتم **حل** المشكلة الأصغر.

3. يتم **دمج** الحل أو **توسيعه** لحل المشكلة الأصلية.

أشكال Decrease and Conquer

1. Decrease by one
2. Decrease by factor
3. Decrease by variable size (random factor)

1. المثال الأول:

لدينا 1000 ورقة مرقمة بحاجة إلى ترتيب.

طريقة الحل

- نصغر المسألة من أجل $m = 5$ (أوراق).
- نحل المسألة المصغرة.
- نحل المسألة من أجل $m + 1 = 6$ وفي الخطوة التالية نحلها من أجل $m + 1 + 1 = 7$ وهكذا خطوة بخطوة حتى $1000 = 1 + 1 + \dots + 1 + m + 1$.

إذاً لحل المسألة من أجل n ورقة نحتاج إلى $n-1$ ورقة مرتبة.

ففي مثالنا، للوصول إلى 1000 ورقة مرتبة نحتاج إلى ترتيب 999 ورقة.

لدينا عدة خوارزميات تستخدم Decrease and Conquer، ومنها:

Insertion Sort

فكرة عملها باختصار:

تخيل أنك ترتب أوراق لعب في يدك:

تبدأ بورقة واحدة (تعتبرها مرتبة).

ثم تأخذ الورقة التالية وتدخلها في المكان الصحيح بالنسبة للأوراق التي بيدك.

تكرر العملية مع كل ورقة حتى تنتهي كل الأوراق.

بنفس الطريقة، خوارزمية Insertion Sort تعمل على:

اعتبار العنصر الأول في المصفوفة مرتباً.

أخذ العنصر التالي ومقارنته بالعناصر التي قبله.

تحريكه إلى مكانه الصحيح بين العناصر السابقة.

تكرار ذلك وتعميمه حتى تنتهي جميع العناصر.

مثال:

ليكن لدينا المصفوفة {1,7,3,2,5,13,8,6}، و لترتيبها تصاعدياً نقوم بالتالي:

```
public class Main {
    public static void main(String[] args) {
        int[] array = {1,7,3,2,5,13,8,6}; //1
        int n = array.length; //2
        for(int i=1; i < n; i++) //3
        {
            int key=array[i]; //4
            int j=i-1; //5
            while(j >= 0 && array[j] > key) // 6
            {
                array[j+1] = array[j]; //7
                j--; //8
            }
            array[j+1] = key; //9
        }
        for(int c:array) //10
            System.out.println(c); //11
    }
}
```



في السطر 4 نقوم بتخزين قيمة العنصر الذي نريد وضعه في مكانه الصحيح بمتغير key.

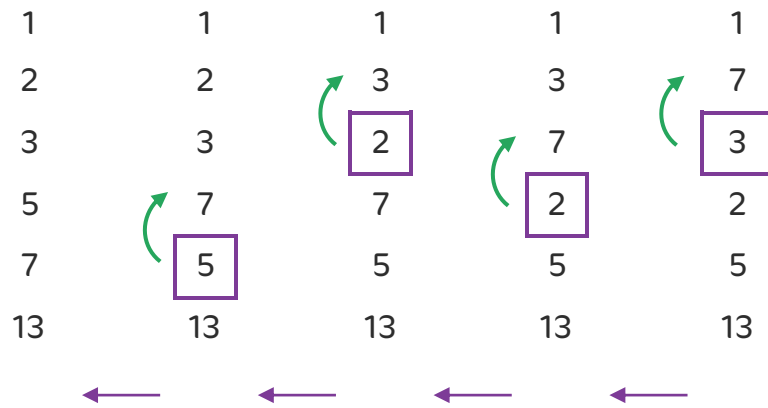
في السطر 5 سيتم تعريف المتغير j الذي سيمثل دور المؤشر حيث أنه سيمر على جميع العناصر التي تسبق key.

في السطر 6 تم إنشاء الحلقة التي سيمر من خلالها المؤشر على العناصر التي تسبق key وتتوقف الحلقة **بإحدى** الحالتين:

1- إما أن يمر المؤشر على عنصر اصغر من ال key فيكون هذا هو الترتيب المناسب لل key.

2- أو ان تنتهي جميع العناصر التي تسبق key و يكون الترتيب المناسب له هو أول عنصر، وطالما أن الشرطين محققين؛ فإنه يلزمنا نقل العنصر الأكبر من ال key خطوة إلى اليمين، وهذه هي وظيفة السطر 7.

السطر 9 يقوم بوضع ال key في ترتيبه الصحيح بعد كسر الحلقة بإحدى حالتي التوقف.



أول عنصرين مرتبين

و هكذا حتى تصبح المصفوفة مرتبة تصاعدياً بالنسبة لكل عناصرها.

السطر	Cost	Time
5	1	$n - 1$
6	2	$n - 1 \leq v \leq \sum_{i=1}^n i$
7	1	v
8	1	v
9	1	$n - 1$

السطر	Cost	Time
1	1	1
2	1	1
3	2	n
4	1	$n - 1$

عند 6: أفضل حالة هي في حال كانت المصفوفة مرتبة $n - 1$ ، و تكون أسوأ حالة في حال كانت المصفوفة مرتبة تنازلياً.

ملاحظة:

$$n - 1 \leq v \leq \sum_{i=1}^n i \rightarrow \sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$v = n - 1$$

$$T_{Best}(n) = 8n - 7$$

$$O(n)$$

$$v = \frac{n(n+1)}{2}$$

$$T_{Worst}(n) = 2n^2 + 7n - 3$$

$$O(n^2)$$

← أحد مميزات هذه الخوارزمية أنه في أحسن حالاتها يكون تعقيدها خطياً.

Selection Sort

- بالعودة إلى مثالنا السابق يمكننا حل المسألة بطريقة أخرى؛ وذلك بطريقة مشابهة حيث نقسم القائمة الى قسمين ، قسم مرتب وآخر غير مرتب، سيكون في البداية القسم المرتب فارغ والقسم غير المرتب يحوي جميع عناصر المصفوفة على الشكل التالي [5, 2, 7, 1, 4] حيث إنه تعتمد خوارزمية الفرز بالاختيار على ترتيب عناصر المصفوفة من خلال البحث المتكرر عن أصغر عنصر في الجزء غير المرتب من المصفوفة، ثم تبديله مع أول عنصر في هذا الجزء.
- تستمر العملية حتى تصبح جميع العناصر مرتبة تصاعدياً.

تعمل الخوارزمية وفق الخطوات التالية:

1. نحدد الجزء الأول من المصفوفة على أنه بداية الجزء غير المرتب، بينما يكون الجزء الذي قبله (الجزء الفارغ) يعتبر مرتباً.
2. في كل دورة تكرارية، نقوم بالبحث داخل الجزء غير المرتب عن أصغر عنصر وتسجيل موقعه (فهرسه).
3. بعد العثور على أصغر عنصر، نقوم بتبديله مع أول عنصر في الجزء غير المرتب.
4. يتم بعد ذلك تحريك حدود الجزء المرتب بمقدار عنصر واحد إلى اليمين، ويصبح هذا العنصر الجديد ضمن الجزء المرتب.
5. تكرر الخطوات السابقة حتى يتم فحص جميع عناصر المصفوفة، وفي النهاية ستكون المصفوفة مرتبة بالكامل.

الجزء الغير مرتب $\rightarrow [5, 2, 7, 1, 4]$ [] $\leftarrow$ الجزء المرتب

1. البحث عن أصغر عنصر و تسجيل موقعه.

[] [5, 2, 7, 1, 4]

2. التبديل مع أول عنصر في الجزء غير المرتب:

$\Rightarrow$ [] [1, 2, 7, 5, 4]

3. نحرك حدود الجزء المرتب بمقدار عنصر واحد إلى اليمين:

$\Rightarrow$ [1] [2, 7, 5, 4]

4. و نكرر الخطوات ذاتها لنصل إلى المصفوفة المرتبة:

[] [1, 2, 4, 5, 7]

```
1) for (int i = 0; i < arr.length - 1; i++) {
```

```
2) int minIndex = i;
```

// البحث عن أصغر عنصر في الجزء الغير مرتب

```
3) for (int j = i + 1; j < arr.length; j++) {
```

```
4) if (arr[j] < arr[minIndex]) {
```

```
5) minIndex = j;
```

```
}
```

```
}
```

// التبدل بين العنصر الأصغر الذي تم إيجاده و أول عنصر في القسم الغير مرتب

```
6) if (minIndex != i) {
```

```
7) int temp = arr[i];
```

```
8) arr[i] = arr[minIndex];
```

```
9) arr[minIndex] = temp;
```

```
}
```

```
}
```

الكود في جميع حالاته نفس التعقيد.

أسوأ حالة: $O(n^2)$

أفضل حالة: $O(n^2)$

ملاحظة:

$\sum_{i=1}^n i$ تمثل مجموع لحدود متتالية حسابية، و نحسب المجموع من القانون $S = n(n + 1)/2$.

تعقيد طريقة Selection Sort في المثال السابق:

لدينا حلقتان متداخلتان كل منهما $O(n)$.

فالتعقيد الكل $O(n^2)$ (بعد حذف الثوابت و تبسيط التابع).



السطر	Time	Cost
1	(n)	1
2	n	1
3	$\sum_{i=1}^n i$	1
4	$\sum_{i=1}^n i$	1
5	$\sum_{i=1}^n i$	1
6	n	1
7	n	1
8	n	1
9	n	1

و الآن نختتم محاضرتنا بمثال بسيط من الواقع عن عناصر شبه مرتبة

ترتيب ال Packets

Packets: عندما ترسل بيانات عبر الإنترنت (مثل تحميل صورة أو زيارة موقع ويب)، يتم تقسيم هذه البيانات الكبيرة إلى أجزاء أصغر يمكن إدارتها، وتسمى كل قطعة منها "حزمة".

تم إرسال Packets للمستقبل..

و لكن عند وصولهم لم يكونوا بالترتيب الصحيح، لكن بفضل التسلسل الزمني للإرسال و تأخر بعضهم عن بعض قليلاً وصلوا شبه مرتبين (ليسوا عشوائيين بشكل كلي)،

و لإعادة ترتيبهم و تكوين الصفحة نستخدم خوارزمية Insertion Sort

التي يعتمد مبدؤها على Decrease and Conquer ... "تصغير المسألة".

تذكروا دوماً... لا يوجد مشكلة بدون حل ... هناك فقط

مسألة تنتظر حلاً إبداعياً من مهندس مبتكر

